



ENSEEIHT

Département Sciences du Numérique

Conception d'un assistant GraphRAG pour le métamodèle Ariane

Graphe de connaissances, procédures documentaires et couche métier

Auteur : BOUFFI Douaa

Entreprise : SNCF Réseau

Encadrant entreprise : BENNANI Mehdi

Encadrant académique : OUEDERNI Meriem

Période : 08/06 – 18/09

Année universitaire 2025–2026

Résumé

Ce projet porte sur la conception d'un assistant capable d'interroger le métamodèle Ariane et des fiches de procédure ferroviaires. La solution combine un graphe de connaissances Neo4j, une recherche documentaire hybride, plusieurs modèles de langage sélectionnables et une application web multi-utilisateur. Le métamodèle, fourni en XMI, est converti en JSON puis représenté sous forme de classes, attributs et relations. Les fiches sont découpées en passages et indexées par recherche plein texte et vectorielle.

Le principal apport est une chaîne GraphRAG contrôlée : les entités de la question sont résolues avant l'appel au modèle, la question est orientée vers la couche UML, documentaire ou métier, puis un contexte borné et traçable est construit. Le système sait également rechercher des chemins multi-sauts UML ou métier. La génération est découplée du retrieval et peut être confiée à Mistral, Gemini ou Groq.

Une couche métier distincte complète le graphe technique. Les 29 fiches ont été traitées par un pipeline d'extraction, de recontrôle et de sélection graduée. La base actuelle contient 255 concepts métier et 347 relations actives : 123 sont confirmées documentairement et 224 sont classées probables. Ces statuts ne constituent pas une certification ferroviaire ; ils rendent explicite le niveau de confiance avant une validation experte ultérieure.

Mots-clés : GraphRAG, Neo4j, graphe de connaissances, Ariane, Mistral, RAG, extraction d'information, traçabilité.

Abstract

This project designs an assistant for querying the Ariane metamodel and railway procedure sheets. It combines a Neo4j knowledge graph, hybrid document retrieval, selectable language-model providers and a multi-user web application. A controlled GraphRAG pipeline routes each question to the UML, documentary or business layer and supports bounded multi-hop paths. All 29 procedure sheets have been processed through extraction, deterministic review and graduated selection. The current global documentary pilot contains 255 business concepts and 347 active relationships, with an explicit distinction between document-confirmed and probable statements.

Keywords : GraphRAG, Neo4j, knowledge graph, retrieval-augmented generation, information extraction, traceability.

Liste des sigles

API	Interface de programmation applicative.
LLM	Grand modèle de langage.
RAG	Génération augmentée par recherche d'information.
GraphRAG	RAG utilisant explicitement un graphe de connaissances.
XMI	Format d'échange de métadonnées fondé sur XML.
RRF	Méthode de fusion de classements par rang réciproque.
UML	Langage de modélisation unifié.

Table des matières

Résumé i

Abstract ii

Liste des sigles iii

1 Introduction 1

1.1 Origine et inspiration du sujet 1

1.2 Problématique 2

1.3 Objectifs 2

1.4 Organisation du rapport 3

2 Fondements théoriques 4

2.1 Graphes de connaissances 4

2.2 RAG 4

2.3 GraphRAG 4

2.4 Embeddings et recherche hybride 5

2.5 LLM et extraction d'information 5

3 Architecture générale 6

3.1 Vue d'ensemble 6

3.2 Composants 7

3.3 Choix d'une base unique à deux couches 7

4 Construction du graphe technique et documentaire 9

4.1 Conversion du XMI 9

4.2 Modèle Neo4j 10

4.3 Documents et chunks . . . 11
4.4 Volumétrie . . . 12

5 Pipeline GraphRAG en ligne 13
5.1 Diagnostic initial . . . 13
5.2 Traitement d'une question . . . 13
5.2.1 Mémoire conversationnelle . . . 13
5.2.2 Résolution des classes . . . 13
5.2.3 Choix du mode . . . 14
5.2.4 Sous-graphe UML . . . 14
5.2.5 Sous-graphe métier . . . 14
5.2.6 Recherche documentaire . . . 14
5.3 Contexte transmis au fournisseur de génération . . . 14
5.4 Exemple . . . 15

6 Pipeline d'extraction du graphe métier 16
6.1 Motivation et modèle . . . 16
6.2 Sélection déterministe des classes candidates . . . 16
6.3 Extraction par le LLM . . . 17
6.4 Contrôles déterministes . . . 17
6.5 Direction et corrections bornées . . . 18
6.6 Consolidation . . . 18
6.7 Cycle des statuts . . . 18
6.8 État réel du corpus . . . 19

7 Application web et mémoire conversationnelle 20
7.1 API . . . 20
7.2 Authentification . . . 20
7.3 Mémoire . . . 20
7.4 Interface . . . 21
7.5 Gestion des erreurs . . . 22

8 Démarche détaillée et implémentation actuelle 23
8.1 Organisation actuelle du dépôt . . . 23

8.2 Deux temporalités : hors ligne et en ligne . . . 24
8.2.1 Préparation hors ligne . . . 24
8.2.2 Traitement en ligne . . . 24
8.3 Conversion XMI et construction UML . . . 25
8.4 Routage explicite des questions . . . 25
8.5 Recherche des chemins multi-sauts . . . 25
8.6 Recherche documentaire hybride . . . 26
8.7 Pipeline métier : de la fiche à Neo4j . . . 26
8.7.1 Extraction contrôlée . . . 26
8.7.2 Validation V3 graduée . . . 26
8.7.3 Rôle des dossiers intermédiaires . . . 27
8.7.4 Import et traçabilité . . . 27
8.8 Construction de la réponse . . . 27
8.9 API, authentification et mémoire . . . 28
8.10 Reproductibilité et commandes . . . 28

9 Validation et résultats 29
9.1 Stratégie de test . . . 29
9.2 Évaluation fonctionnelle du GraphRAG . . . 29
9.3 Pilote métier . . . 30
9.4 Discussion . . . 30
9.5 Limites . . . 30
9.6 Prototype de reranking . . . 30

10 Conclusion et perspectives 32

A Fichiers, commandes et captures 33
A.1 Fichiers principaux . . . 33
A.2 Commandes utiles . . . 34
A.3 Requêtes Neo4j . . . 34
A.4 Captures à ajouter . . . 34

Table des figures

3.1 Architecture globale du projet 6
3.2 Architecture réelle de l'application GraphRAG Arianes 8

4.1 Extrait du fichier XMI décrivant une classe Arianes 9
4.2 Représentation JSON obtenue après conversion 10
4.3 Sous-graphe Neo4j de la classe Ligne 11
4.4 Résultat des requêtes de comptage et vérification des embeddings 12

5.1 Réponse GraphRAG distinguant le graphe UML, le graphe métier et les sources 15

6.1 Exemple de sous-graphe métier autour du poste d'aiguillage 19

7.1 Écran d'inscription ou de connexion 21
7.2 Interface principale et historique des conversations 21

Liste des tableaux

3.1 Responsabilités des composants principaux 7

4.1 Relations principales du graphe UML 10

4.2 État observé du graphe technique 12

9.1 État actuel de la couche métier dans Neo4j 30

A.1 Inventaire fonctionnel 33

Chapitre 1

Introduction

1.1 Origine et inspiration du sujet

La transformation numérique de SNCF Réseau nécessite de rendre accessibles des connaissances ferroviaires nombreuses, hétérogènes et produites par des métiers différents. Ces connaissances sont présentes à la fois dans des modèles structurés, tels que le métamodèle ARIANE, et dans des documents décrivant les règles, les équipements et les procédures métier. Leur dispersion rend cependant leur consultation et leur mise en relation difficiles pour un utilisateur qui ne maîtrise pas la structure du système d'information.

Ce sujet de stage s'inscrit dans la continuité des travaux d'Alexis Chartrain consacrés au jumeau numérique du système ferroviaire [1]. Ces travaux mettent en évidence la nécessité de construire une représentation globale, systémique, commune et partagée du domaine ferroviaire. Une telle représentation doit permettre aux différents acteurs de partager une même compréhension des concepts et favoriser l'interopérabilité sémantique entre les composants du système d'information de SNCF Réseau.

La thèse présente la modélisation de domaine comme un levier essentiel pour organiser les connaissances du système ferroviaire. Elle s'appuie notamment sur le modèle de référence ARIANE, sur l'ingénierie dirigée par les modèles et sur les ontologies. Elle propose plusieurs représentations complémentaires de la connaissance : un glossaire en langage naturel, un thésaurus, un modèle conceptuel orienté objet exprimé en UML et une ontologie formalisée en OWL. Ces représentations permettent de définir les concepts du domaine ainsi que leurs relations taxonomiques et sémantiques.

Le présent stage reprend cette idée de représentation structurée de la connaissance, mais l'aborde sous l'angle de son exploitation par un agent conversationnel. L'objectif n'est pas de reconstruire l'ontologie proposée dans la thèse ni de réaliser le jumeau numérique lui-même. Il consiste à développer un prototype permettant d'interroger en langage naturel le métamodèle ARIANE et les fiches documentaires associées.

Pour cela, les classes, les attributs, les relations et les héritages du métamodèle ARIANE sont transformés en un graphe de connaissances Neo4j. Les fiches de procédure sont découpées en passages documentaires indexés. Une couche métier complémentaire représente les concepts et les relations extraits de ces documents, tout en conservant un lien vers les passages qui les justifient. L'ensemble est interrogé selon une approche GraphRAG, dans

laquelle la recherche combine la structure du graphe et le contenu des documents avant de transmettre un contexte contrôlé à un modèle de langage.

Ainsi, la thèse fournit le cadre conceptuel du stage : partager un vocabulaire ferroviaire commun, préserver les relations entre les concepts et rendre les connaissances interopérables. Le prototype développé pendant le stage constitue une expérimentation applicative de ces principes. Il cherche à rendre la connaissance structurée plus facilement accessible, sans exiger de l'utilisateur qu'il maîtrise le métamodèle UML, le langage Cypher ou l'organisation interne des documents.

1.2 Problématique

À partir de ce contexte, la problématique du stage peut être formulée de la manière suivante :

> *Comment rendre les connaissances structurelles et documentaires du domaine ferroviaire accessibles en langage naturel, tout en conservant les relations sémantiques du métamodèle ARIANE et la traçabilité vers les documents métier ?*

Cette problématique impose plusieurs exigences :

- représenter les relations du métamodèle dans Neo4j ;
- retrouver des passages même lorsque la question n'emploie pas exactement les mêmes mots ;
- distinguer une relation UML d'une connaissance fonctionnelle issue d'une fiche ;
- empêcher l'import automatique de relations métier incertaines ;
- conserver la source de chaque information ;
- isoler l'historique de chaque utilisateur et de chaque conversation.

1.3 Objectifs

Les objectifs opérationnels sont :

1. convertir les sources XMI en données structurées ;
2. construire un graphe technique du métamodèle ;
3. découper et indexer les fiches documentaires ;
4. mettre en place un retrieval GraphRAG déterministe ;
5. développer une application web avec mémoire et authentification ;

6. extraire une couche de concepts métier contrôlée ;
7. évaluer la solution par des tests reproductibles.

1.4 Organisation du rapport

Après les fondements théoriques, le rapport présente l'architecture générale, la construction du graphe technique, le fonctionnement du GraphRAG, puis le pipeline métier. Les derniers chapitres décrivent l'application, les résultats, les limites et les perspectives.

Point de vigilance. Les termes « extraction », « validation » et « import » désignent trois étapes distinctes. Une relation extraite par un LLM n'est pas considérée comme validée et n'est pas nécessairement présente dans Neo4j.

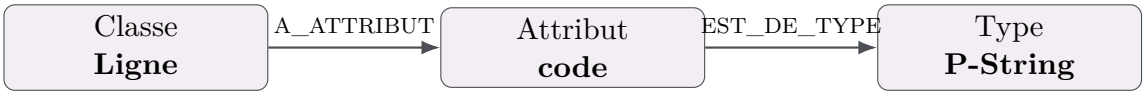
Chapitre 2

Fondements théoriques

2.1 Graphes de connaissances

Un graphe de connaissances représente l'information par des nœuds et des relations. Dans Neo4j, chaque nœud possède un ou plusieurs labels et des propriétés. Une relation est orientée, typée et peut également porter des propriétés.

Par exemple :



Cette représentation facilite le parcours des dépendances, l'explication d'un chemin et la limitation d'une recherche à un voisinage pertinent.

2.2 RAG

La génération augmentée par recherche, ou RAG, sépare la connaissance externe de la génération de texte. Une question est d'abord utilisée pour retrouver des documents pertinents. Ces documents sont ensuite insérés dans le contexte d'un modèle de langage, qui rédige la réponse [3].

$$\text{Question} \rightarrow \text{Recherche} \rightarrow \text{Contexte} \rightarrow \text{LLM} \rightarrow \text{Réponse} \tag{2.1}$$

Le modèle n'a donc pas besoin d'avoir mémorisé les données Arianes pendant son entraînement.

2.3 GraphRAG

GraphRAG ajoute une étape de recherche structurée dans un graphe. Le simple stockage de données dans Neo4j ne suffit pas : l'agent doit résoudre les entités de la question,

parcourir les relations utiles et transmettre ce sous-graphe au modèle [2].

Dans ce projet, le contexte final peut combiner :

— un sous-graphe UML ;

— un sous-graphe métier validé ;

— des passages documentaires ;

— l'historique récent de la conversation.

2.4 Embeddings et recherche hybride

Un modèle d'embedding transforme un texte en vecteur numérique. La proximité sémantique est mesurée par la similarité cosinus :

$$\mathrm{sim}(q, d) = \frac{q \cdot d}{\|q\|\|d\|}. \tag{2.2}$$

La recherche vectorielle est utile pour les reformulations. La recherche plein texte reste meilleure pour des noms ou acronymes exacts. Le système combine les classements avec une fusion par rang réciproque :

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rang}_r(d)}. \tag{2.3}$$

Cette association limite la dépendance à un unique mécanisme de recherche. Les principes probabilistes de recherche lexicale sont notamment décrits dans les travaux sur BM25 [4].

2.5 LLM et extraction d'information

Mistral est utilisé pour les embeddings et pour l'extraction structurée hors ligne. La génération des réponses peut être confiée à Mistral, Gemini ou Groq selon le choix de l'utilisateur. Aucun de ces modèles n'est un système spécialisé de reconnaissance d'entités Arianes : les sorties d'extraction doivent donc être contrôlées par du code déterministe avant toute utilisation.

Chapitre 3

Architecture générale

3.1 Vue d'ensemble

L'architecture sépare les traitements hors ligne, réalisés lors de la préparation des connaissances, et le traitement en ligne, exécuté à chaque question.

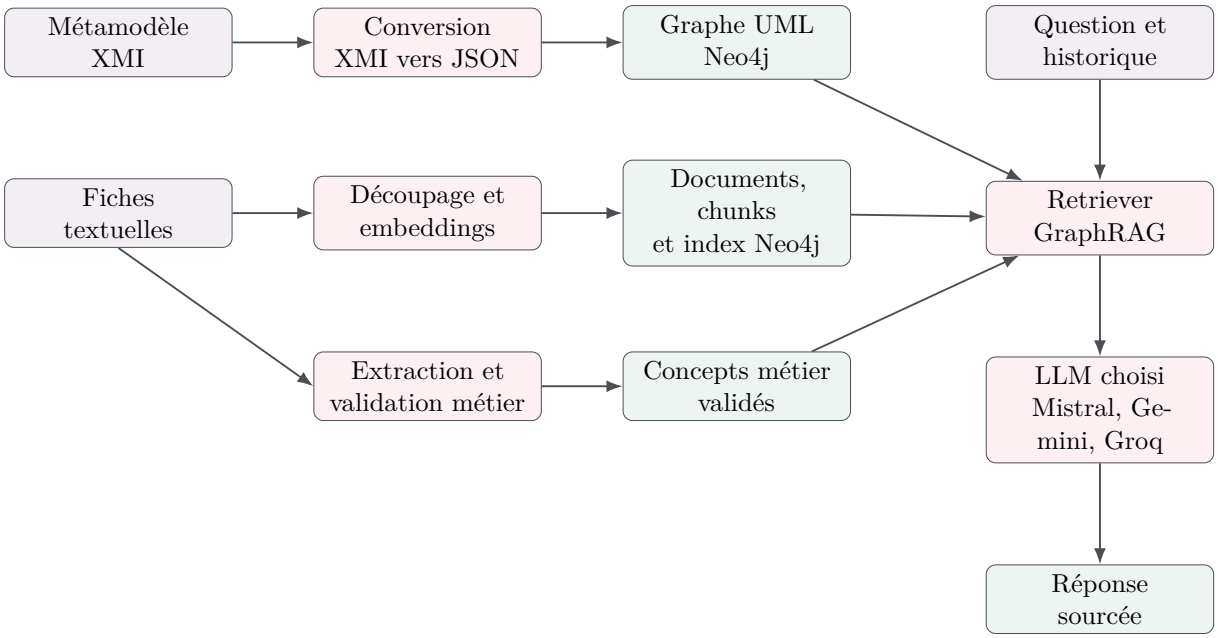


FIGURE 3.1 – Architecture globale du projet

3.2 Composants

TABLE 3.1 – Responsabilités des composants principaux

Composant	Responsabilité
`arianes/pipelines/xmi_to_json.py`	Convertir le métamodèle XMI en JSON exploitable.
`arianes/graph/uml_builder.py`	Construire le graphe UML, importer et indexer les documents.
`arianes/retrieval.py`	Router les questions et sélectionner sous-graphes, chemins et passages.
`arianes/agent.py`	Construire le prompt final et appeler le fournisseur choisi.
`api.py`	Authentification, conversations, messages et routes HTTP.
`arianes/pipelines/extraction_metier.py`	Proposer les concepts et relations métier.
`arianes/pipelines/validation_metier.py`	Appliquer le contrôle gradué V3.
`arianes/graph/metier_builder.py`	Importer la sélection documentaire globale.

3.3 Choix d'une base unique à deux couches

Le graphe technique et le graphe métier sont conservés dans la même base afin de permettre leur traversal conjoint. Ils restent cependant séparés par leurs labels et leurs relations : une `Classe` UML n'est jamais fusionnée avec un `ConceptMetier`. Le pont explicite `REPRESENTE_PAR` maintient la distinction entre représentation technique et sens fonctionnel.

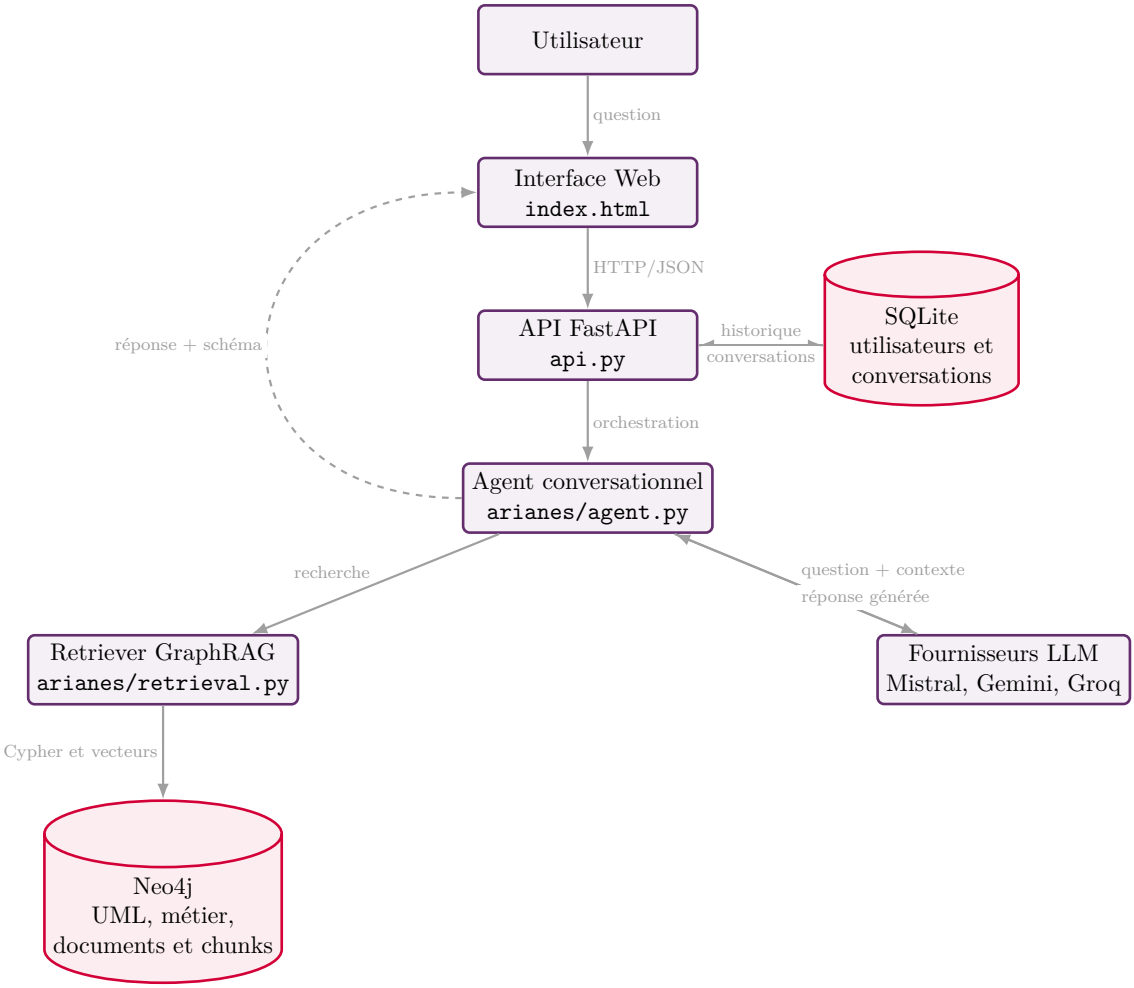


FIGURE 3.2 – Architecture réelle de l'application GraphRAG Arianes

Chapitre 4

Construction du graphe technique et documentaire

4.1 Conversion du XMI

Le format XMI contient les définitions du métamodèle mais reste difficile à inspecter et à importer directement. Le script `arianes/pipelines/xmi_to_json.py` analyse les éléments XML et produit des fichiers JSON contenant les classes, leurs attributs, leurs types et leurs relations. Cette représentation intermédiaire facilite les contrôles et découple le parseur XMI du constructeur Neo4j.

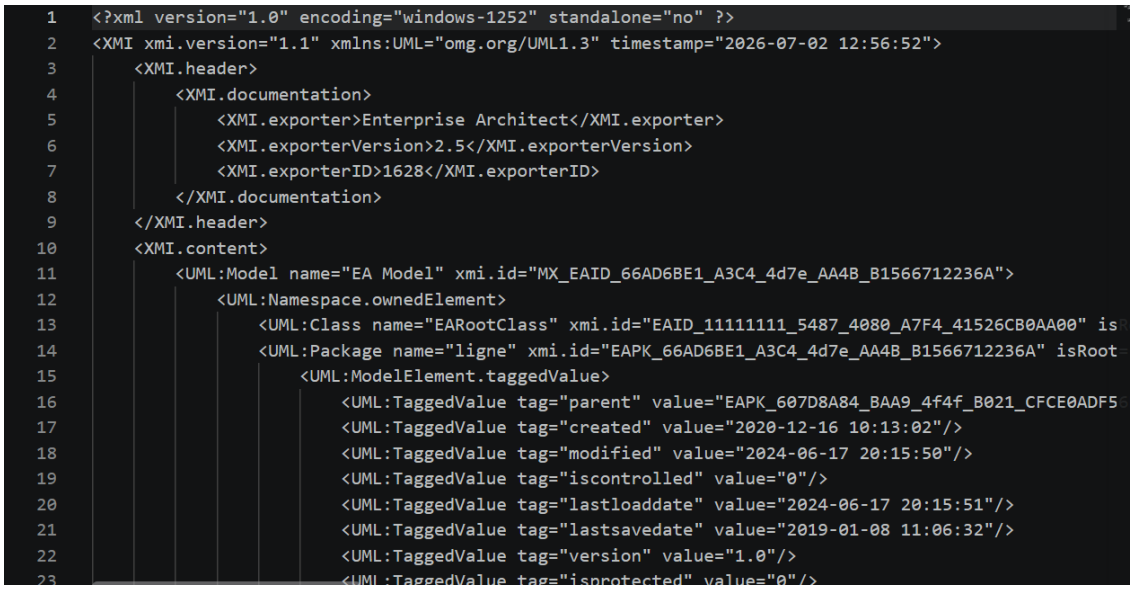


FIGURE 4.1 – Extrait du fichier XMI décrivant une classe Arianes



```
{
    "enums": [],
    "classes": [
        {
            "nom": "RessourceAbstraite",
            "abstraite": false,
            "description": "Élément externe (Class)",
            "attributs": []
        },
        {
            "nom": "CompetenceSurLigne",
            "abstraite": false,
            "description": "Élément externe (Class)",
            "attributs": []
        },
        {
            "nom": "ElementTopologiqueLineaire",
            "abstraite": false,
            "description": "Élément externe (Class)",
            "attributs": []
        },
        {
            "nom": "PointOrientation",
```

FIGURE 4.2 – Représentation JSON obtenue après conversion

4.2 Modèle Neo4j

Le constructeur crée notamment les labels `Classe`, `Attribut`, `Type`, `Enum` et `ValeurEnum`. Les relations structurantes sont présentées dans le tableau 4.1.

TABLE 4.1 – Relations principales du graphe UML

Relation	Signification
`A_ATTRIBUT`	Une classe possède un attribut.
`EST_DE_TYPE`	Un attribut possède un type.
`HERITE_DE`	Une classe hérite d'une autre.
`ASSOCIE_A`	Deux classes sont associées.
`COMPOSE_DE`	Composition entre classes.
`AGREGE`	Agrégation entre classes.
`A_VALEUR`	Une énumération contient une valeur.



FIGURE 4.3 – Sous-graphe Neo4j de la classe Ligne

4.3 Documents et chunks

Chaque fiche devient un nœud `Document`. Son contenu est découpé en nœuds `Chunk`, reliés par `APPARTIENT_A`. Le découpage produit des unités suffisamment courtes pour une recherche précise, tout en conservant l'ordre du passage dans le document.

Chaque chunk possède un embedding Mistral de dimension 1024. Un index vectoriel et un index plein texte permettent une recherche hybride. Lors de l'audit, les 244 chunks disposaient tous d'un embedding.

4.4 Volumétrie

TABLE 4.2 – État observé du graphe technique

Élément	Nombre	Élément	Nombre
Classes	998	Attributs	738
Documents	33	Chunks	244
Énumérations	47	Valeurs d'énumération	146
Types	7	Embeddings de chunks	244

Ces chiffres décrivent l'état testé de la base. Ils doivent être recalculés après une reconstruction du graphe.

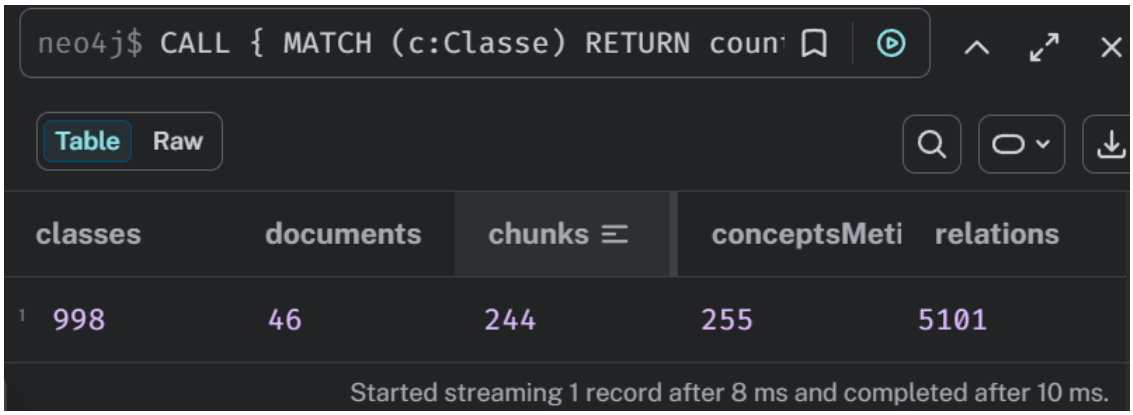


FIGURE 4.4 – Résultat des requêtes de comptage et vérification des embeddings

Chapitre 5

Pipeline GraphRAG en ligne

5.1 Diagnostic initial

L'analyse du système a montré que le graphe technique était globalement construit, mais que l'agent ne l'exploitait pas encore par une chaîne de retrieval suffisamment contrôlée. La correction a consisté à séparer clairement le retrieval déterministe de la génération. Le fournisseur de génération n'est pas chargé de décider librement quelles données lire dans Neo4j.

5.2 Traitement d'une question

Pour une question q, le traitement peut être résumé par :

$$q \to E(q) \to M(q) \to \Big(G_{UML}, G_{métier}, D\Big) \to C \to LLM, \tag{5.1}$$

où $E(q)$ représente les entités résolues, $M(q)$ le mode de recherche, G les sous-graphes, D les passages et C le contexte final.

5.2.1 Mémoire conversationnelle

Les derniers messages utilisateur sont ajoutés à la question courante. Une question telle que « et quels sont ses attributs ? » peut ainsi reprendre une classe mentionnée précédemment, sans mélanger les conversations d'autres utilisateurs.

5.2.2 Résolution des classes

Le catalogue des classes est chargé depuis Neo4j. Les noms sont normalisés : accents, casse et CamelCase sont traités. Le retriever combine correspondance exacte, recouvrement des mots et similarité de chaîne. Les classes courtes ou trop génériques sont filtrées afin de réduire les faux positifs.

5.2.3 Choix du mode

Six modes sont distingués :

STRUCTURE attributs, héritages, associations et compositions ;
DOCUMENTAIRE procédures, règles, définitions et fonctionnement ;
METIER concepts et relations fonctionnelles issus des fiches ;
CHEMIN_UML plus court chemin entre deux classes UML ;
CHEMIN_METIER plus court chemin entre deux concepts métier ;
HYBRIDE utilisation simultanée du graphe et des fiches.

5.2.4 Sous-graphe UML

Le système extrait un voisinage borné des classes racines. Il ne transmet jamais l'ensemble des 998 classes au modèle. Cette limitation réduit le bruit, le nombre de tokens et le risque d'interpréter une relation hors sujet.

5.2.5 Sous-graphe métier

Seuls les nœuds et relations portant `valide=true` sont utilisés. Les concepts sont sélectionnés grâce à leur nom, leurs alias et leurs classes Arianes associées. Les relations fournissent leurs citations, conditions et identifiants de chunks.

5.2.6 Recherche documentaire

En mode documentaire ou hybride, l'embedding de la question est comparé aux chunks. Les résultats vectoriels et plein texte sont fusionnés par RRF. Les chunks directement désignés comme preuves d'une relation métier sont ajoutés prioritairement et dédupliqués.

5.3 Contexte transmis au fournisseur de génération

Le contexte contient explicitement :

Listing 5.1 – Structure simplifiée du contexte GraphRAG

```
{
  "mode": "HYBRIDE",
  "classes_racines": ["Poste"],
  "sous_graphe": {"classes": [], "relations": []},
  "graphe_metier": {"concepts": [], "relations": []},
  "chunks": [],
  "citations": []
}
```

Le prompt demande de ne pas inventer de classe, relation ou procédure, de distinguer les faits UML des connaissances métier et de respecter les conditions portées par les relations.

5.4 Exemple

Pour « Quels équipements sont commandés par un poste d'aiguillage ? », le retriever sélectionne le concept `poste_aiguillage`, les relations `EST_COMMANDE_PAR`, puis leurs passages de preuve. Le fournisseur sélectionné formule enfin une réponse en distinguant connaissance métier et association UML.

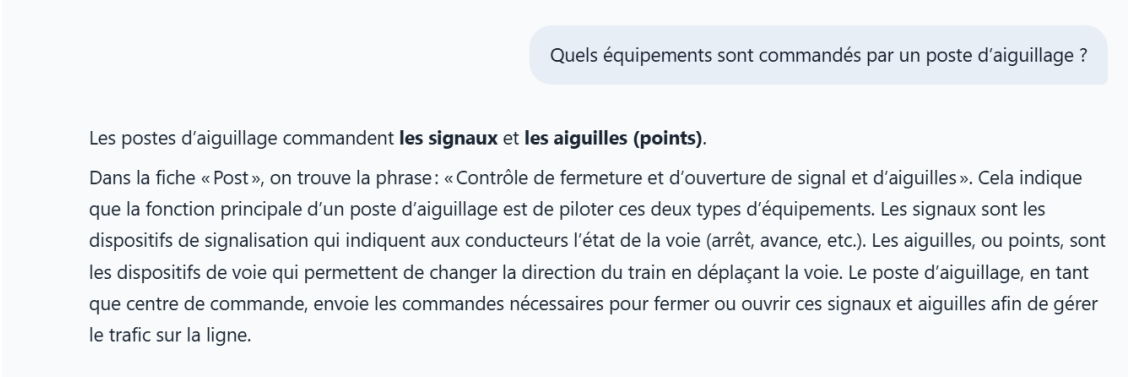


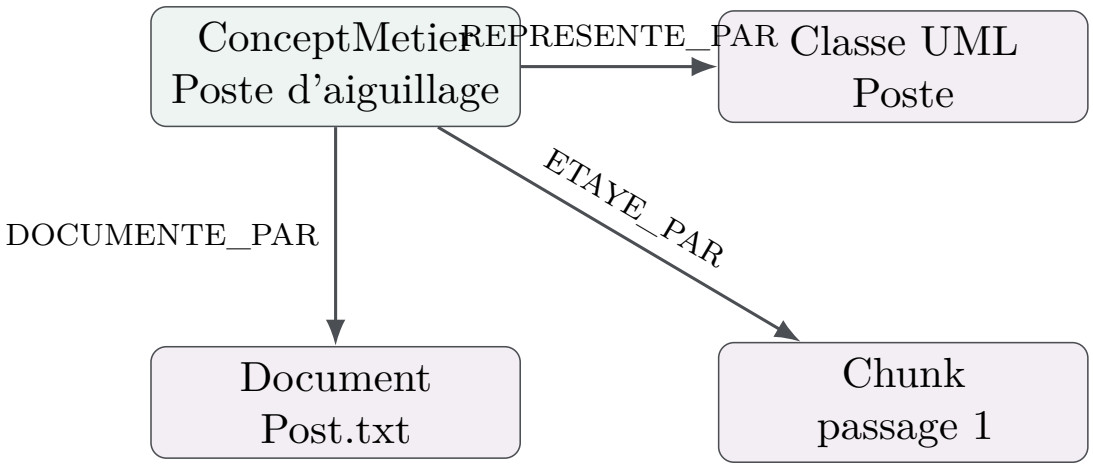
FIGURE 5.1 – Réponse GraphRAG distinguant le graphe UML, le graphe métier et les sources

Chapitre 6

Pipeline d'extraction du graphe métier

6.1 Motivation et modèle

Les fiches expriment des connaissances fonctionnelles qui ne figurent pas toujours comme relations UML. La couche métier introduit des nœuds `ConceptMetier` séparés des classes. Les principaux liens sont :



6.2 Sélection déterministe des classes candidates

Le LLM ne reçoit pas les 998 classes. Pour chaque passage, la fonction `classes_candidates` charge le catalogue JSON et normalise le texte. Une classe reçoit un score de 1 si son nom est présent exactement. Sinon, le score correspond au recouvrement entre les mots du passage et ceux du nom de la classe :

$$score(c, p) = \frac{|mots(c) \cap mots(p)|}{|mots(c)|}. \tag{6.1}$$

Les classes dont le score atteint 0,5 sont triées et les 30 premières sont transmises à Mistral. Pour chacune, le prompt fournit le nom, jusqu'à 300 caractères de description et 15 attributs. La sélection actuelle est lexicale ; elle pourrait être améliorée par une segmentation systématique du CamelCase et par des synonymes contrôlés.

6.3 Extraction par le LLM

Le passage, ses lignes et les classes candidates sont insérés dans un prompt d'extraction. Mistral doit retourner exclusivement un JSON contenant :

- les concepts métier et leurs alias ;
- une catégorie parmi équipement, objet, type, acteur, document, procédure, localisation, date, valeur, caractéristique ou système ;
- les correspondances exactes vers les classes candidates ;
- des triplets source–prédicat–cible ;
- une citation exacte pour chaque relation.

Il ne s'agit pas d'un modèle NER spécialisé : les entités sont identifiées par les capacités générales du LLM, guidées par le schéma de sortie et des exemples positifs et négatifs.

Listing 6.1 – Sortie attendue de l'extracteur

```
{
  "concepts": [
    {"id_local": "poste_aiguillage",
     "nom": "Poste d'aiguillage",
     "categorie": "EQUIPEMENT",
     "classes_arianes": ["Poste"]}
  ],
  "relations": [
    {"source": "signal",
     "type": "EST_COMMANDE_PAR",
     "cible": "poste_aiguillage",
     "citation_exacte": "..."}
  ]
}
```

6.4 Contrôles déterministes

La sortie n'est jamais écrite directement dans Neo4j. Le code vérifie successivement :

1. la validité du JSON et des identifiants normalisés ;

2. l'existence exacte des classes Arianes proposées ;
3. l'existence des concepts source et cible ;
4. l'absence d'auto-relation ;
5. l'appartenance du prédicat à une liste contrôlée ;
6. la présence textuelle de la citation dans le passage ;
7. la compatibilité de certaines catégories avec le prédicat ;
8. la présence d'indices lexicaux justifiant le prédicat ;
9. les négations et modalités telles que « peut être » ou « muni ou non ».

Une absence d'indice lexical produit un avertissement plutôt qu'un rejet automatique. La validation V3 classe ensuite la relation comme `VALIDE`, `PROBABLE` ou `REJETE` selon ses preuves et son score.

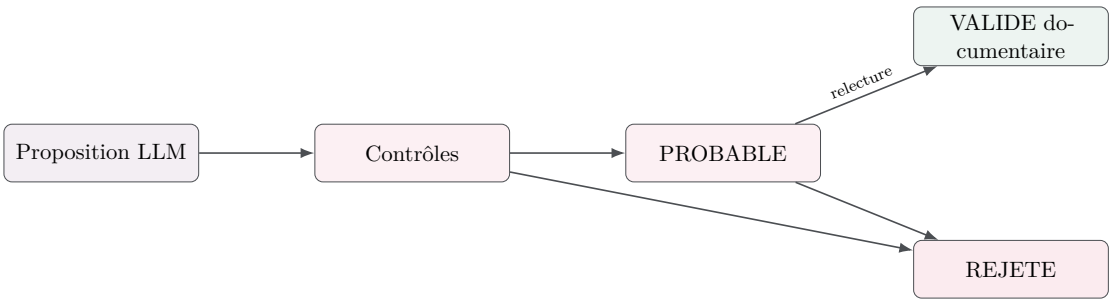
6.5 Direction et corrections bornées

La direction d'un triplet est stricte : « A commande B » donne `A COMMANDE B`, tandis que « B est commandé par A » donne `B EST_COMMANDE_PAR A`. Pour `EST_UN`, le type spécialisé est la source. Les inversions ne sont corrigées que pour des cas relus et accompagnés de la citation attendue. Chaque correction conserve l'ancien triplet et son motif.

6.6 Consolidation

Les concepts extraits de plusieurs passages sont fusionnés par nom normalisé. Les alias, classes et documents sont réunis. Une incompatibilité de catégories conduit à `INCONNU`. Les extrémités des relations sont ensuite réécrites vers les identifiants canoniques et les auto-relations sont contrôlées une seconde fois. Deux preuves différentes d'un même triplet restent distinctes.

6.7 Cycle des statuts



Les objets `VALIDE` et `PROBABLE` sans motif bloquant peuvent être importés. Leur statut reste attaché à la relation et contrôle la prudence de la réponse.

6.8 État réel du corpus

Les 29 fiches ont été traitées. Le recontrôle V3 a observé 345 concepts, retenu 117 relations `VALIDE` et 224 relations `PROBABLE`, et isolé 24 relations `REJETE`. Trente-six relations retenues ont bénéficié d'une correction bornée.

Point de vigilance. Le pilote documentaire global est désormais importé. La base observée contient 255 concepts métier et 347 relations actives : 123 `VALIDE` et 224 `PROBABLE`. Le statut `VALIDE` reste documentaire et non expert.

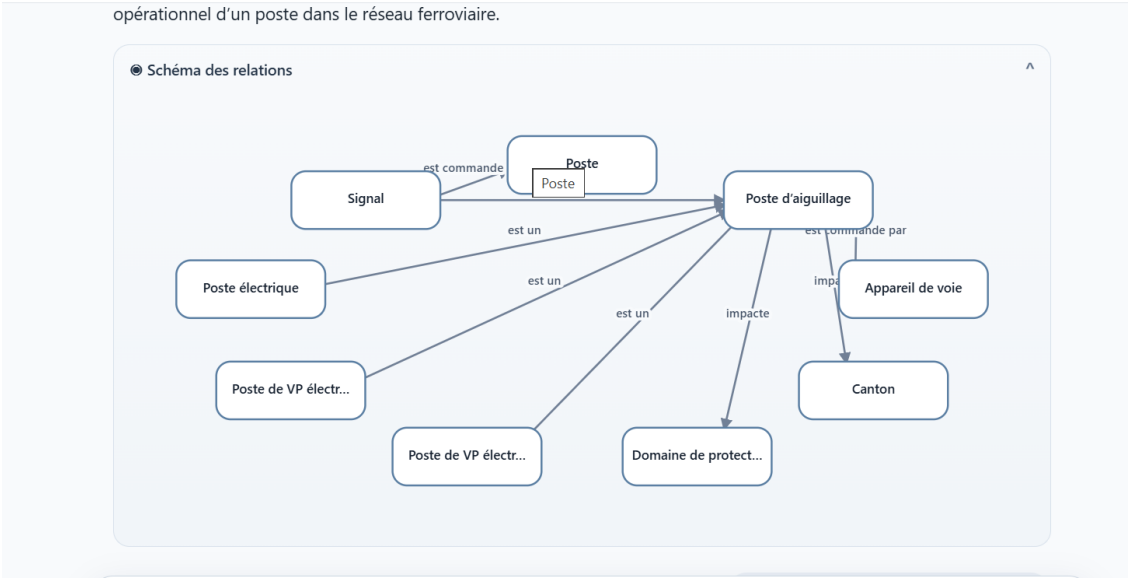


FIGURE 6.1 – Exemple de sous-graphe métier autour du poste d'aiguillage

Chapitre 7

Application web et mémoire conversationnelle

7.1 API

L'application repose sur FastAPI. L'API expose les opérations d'inscription, de connexion, de création et de consultation des conversations ainsi que l'envoi d'une question à l'agent. Elle partage le driver Neo4j et l'agent afin d'éviter plusieurs instances incohérentes. Le fournisseur de génération est transmis explicitement avec la question.

7.2 Authentification

Les utilisateurs sont enregistrés dans SQLite. Les mots de passe ne sont pas stockés en clair : une dérivation PBKDF2-HMAC-SHA256 avec sel et 600 000 itérations est utilisée. Les sessions sont associées à des cookies. Chaque route contrôlant une conversation vérifie son propriétaire.

Les clés Mistral, Gemini et Groq ainsi que les identifiants Neo4j sont chargés depuis `.env`, exclu de Git. Le fichier `.env.example` documente les variables attendues sans contenir de secret.

7.3 Mémoire

Le graphe de connaissances et la mémoire conversationnelle remplissent des fonctions différentes. Neo4j contient les connaissances Arianes ; SQLite conserve les échanges propres à un utilisateur. Les derniers messages sont transmis à l'agent afin de résoudre les reprises telles que « et ses attributs ? ».

7.4 Interface

L'interface permet de se connecter, de créer une conversation, de sélectionner une conversation existante, de choisir le fournisseur de génération, d'afficher le schéma associé à une réponse et de dialoguer avec l'agent. Elle communique avec l'API sans accéder directement à Neo4j ou aux fournisseurs.

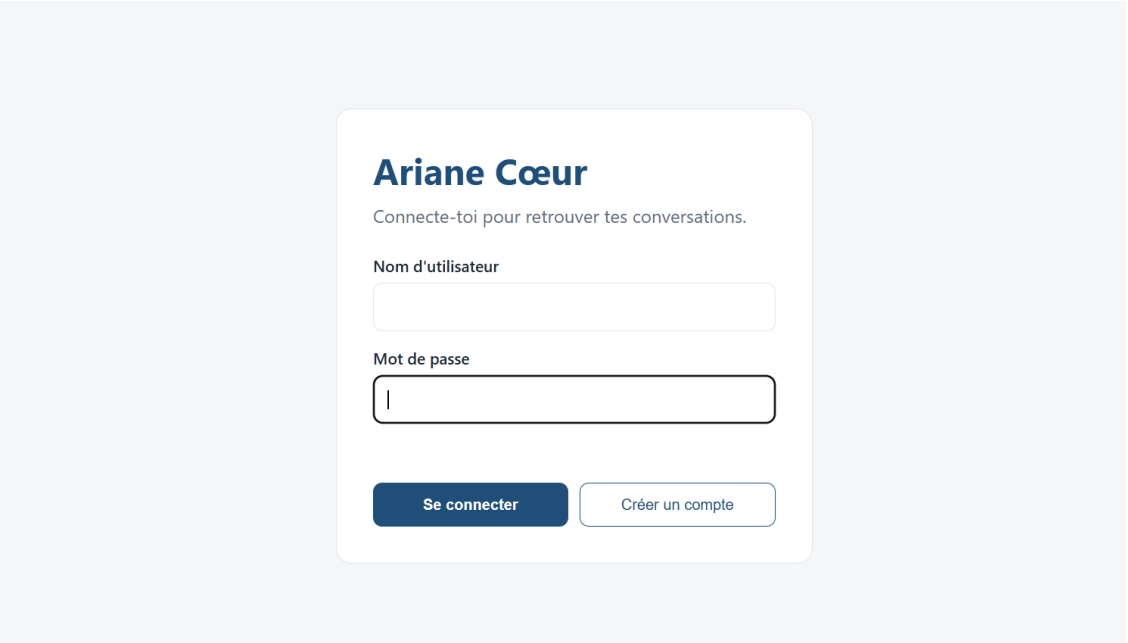


FIGURE 7.1 – Écran d'inscription ou de connexion

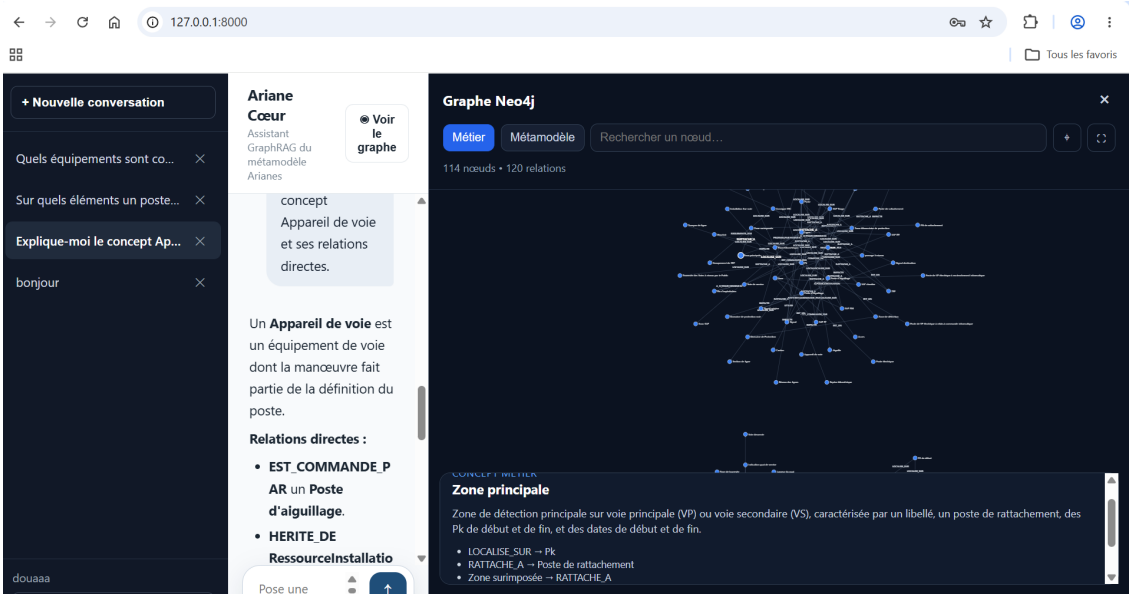


FIGURE 7.2 – Interface principale et historique des conversations

7.5 Gestion des erreurs

Les indisponibilités de Neo4j et des fournisseurs sont distinguées. Une erreur 429 de quota, une erreur 413 de contexte ou une erreur réseau ne doit pas être présentée comme une absence du graphe. Pour une démonstration, il reste prudent de conserver des captures des réponses validées en cas de problème de proxy ou de réseau.

Chapitre 8

Démarche détaillée et implémentation actuelle

Ce chapitre décrit la version effectivement présente dans le dépôt. Il complète les vues fonctionnelles précédentes par une lecture reproductible des traitements, des fichiers intermédiaires et des décisions de conception. La règle directrice est de séparer trois responsabilités : construire la connaissance, sélectionner les éléments pertinents, puis rédiger une réponse. Un LLM ne doit pas remplir simultanément ces trois rôles.

8.1 Organisation actuelle du dépôt

Le dépôt a été réorganisé afin de distinguer le code actif, les données, les essais et les anciens prototypes. Le point d'entrée reste `api.py` ; le code applicatif réutilisable est regroupé dans le package `arianes/`.

Listing 8.1 – Arborescence fonctionnelle actuelle

```
api.py API FastAPI et point d'entree
index.html interface web
arianes/
  agent.py orchestration et generation
  retrieval.py routage et retrieval GraphRAG
  paths.py chemins centralises
  evaluation.py evaluation fonctionnelle
  graph/
    uml_builder.py import UML et documents
    metier_builder.py import de la couche metier
  pipelines/
    xmi_to_json.py conversion XMI vers JSON
    extraction_metier.py extraction d'une fiche
    extraction_globale.py traitement du corpus
    validation_metier.py validation V3
    recontrole_metier.py second controle hors ligne
    selection_prudente.py selection globale graduee
    generer_pilotes.py pilote par document
    normalisation_metier.py fusion de doublons
```

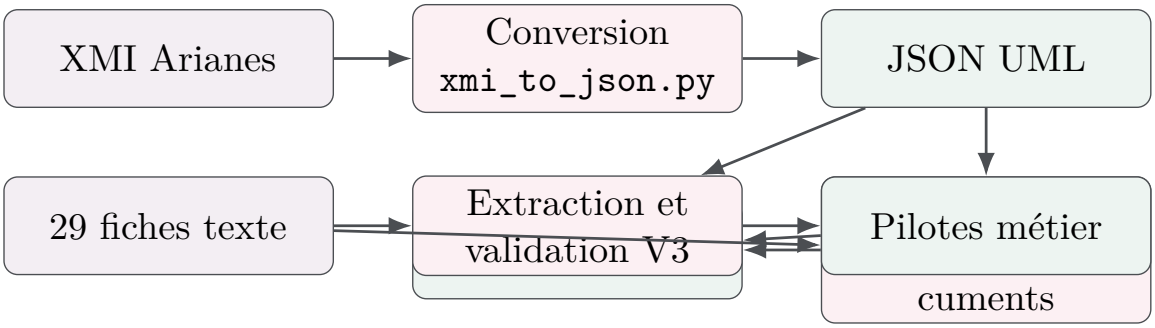
```
project_data/
  xmi/ sources du metamodele
  uml/ JSON structures
  procedures/ fiches texte
  metier/ propositions et pilotes
tests/ tests automatises
ml/ prototype de reranking
documentation/ documentation technique
legacy/ anciens prototypes non actifs
```

Le module `arianes/paths.py` centralise les chemins. Cette décision évite que chaque script reconstruise des chemins relatifs différents et rend les commandes exécutables depuis la racine. Le dossier `legacy/` est conservé pour la traçabilité, mais il ne participe pas au lancement normal.

8.2 Deux temporalités : hors ligne et en ligne

8.2.1 Préparation hors ligne

La préparation hors ligne est exécutée lorsque les sources changent. Elle comprend la conversion XMI, la construction du graphe UML, le découpage des fiches, le calcul des embeddings, puis l'extraction et la validation du graphe métier. Ces opérations sont plus coûteuses, mais leurs résultats sont persistés dans Neo4j et dans des fichiers JSON auditables.



8.2.2 Traitement en ligne

Le traitement en ligne commence à la réception d'une question. Il ne reconstruit pas le graphe. Il consulte les index et les relations déjà préparés, produit un contexte compact, puis appelle le fournisseur choisi par l'utilisateur. La séparation garantit qu'un échec du fournisseur de génération ne modifie jamais la connaissance stockée.

8.3 Conversion XMI et construction UML

`arianes/pipelines/xmi_to_json.py` analyse les exports XMI avec leur espace de noms. Il extrait les classes, descriptions, attributs, multiplicités, énumérations, généralisations et associations. Une étape de déduplication stabilise les listes lorsqu'un export contient plusieurs représentations du même élément. Le JSON obtenu joue le rôle de contrat entre le format externe XMI et Neo4j : le builder n'a pas besoin de connaître les détails XML.

`arianes/graph/uml_builder.py` importe ensuite les objets par `MERGE`. Les contraintes d'unicité portent notamment sur `Classe.nom`, `Attribut.key`, `Document.id` et `Chunk.id`. Une classe possède ses attributs par `A_ATTRIBUT` ; un attribut pointe vers son type par `EST_DE_TYPE`. Les relations entre classes sont limitées à `HERITE_DE`, `ASSOCIE_A`, `COMPOSE_DE` et `AGREGE`.

Chaque fiche textuelle devient un `Document`. Elle est découpée par un séparateur de texte avec chevauchement afin d'éviter de couper brutalement une information à la frontière de deux passages. Chaque `Chunk` conserve son texte, sa position et son embedding. Deux index complémentaires sont créés : un index plein texte pour les termes précis et un index vectoriel pour les reformulations sémantiques.

8.4 Routage explicite des questions

`arianes/retrieval.py` commence par normaliser la question : suppression des accents, passage en minuscules et découpage des noms CamelCase. Il résout ensuite les classes par correspondance exacte, recouvrement lexical et similarité de chaîne. Les entités ambiguës ne sont pas remplacées arbitrairement. Par exemple, *Habillage* ne désigne aucune classe exacte ; le système propose alors `HabillageParcours`, `HabillageValeur` ou d'autres classes réelles et demande une précision.

Le routage distingue actuellement six modes :

STRUCTURE consultation des classes, attributs et relations UML ;
DOCUMENTAIRE recherche de définitions, règles ou étapes dans les chunks ;
METIER interrogation des concepts et relations issus des fiches ;
CHEMIN_UML plus court chemin multi-sauts entre deux classes UML ;
CHEMIN_METIER plus court chemin entre deux concepts métier actifs ;
HYBRIDE combinaison ciblée de plusieurs couches lorsque la formulation le justifie.

Le qualificatif explicite de l'utilisateur est prioritaire. Ainsi, « chemin UML » ne doit jamais être redirigé vers les relations métier. Cette distinction a été introduite après l'observation d'un cas où *Habillage* et *Ligne* étaient recherchés dans la mauvaise couche.

8.5 Recherche des chemins multi-sauts

Pour un chemin UML, deux classes exactes sont résolues avant la requête. Neo4j recherche le plus court chemin, borné à huit relations, parmi les quatre types structurels autorisés.

Le parcours est non orienté pour découvrir une connexion, mais la réponse conserve le sens réel de chaque relation. Lorsqu'une arête est parcourue à l'envers, ce fait est signalé. Cette précaution évite de présenter une association inversée comme une nouvelle règle.

Pour un chemin métier, la profondeur est bornée à cinq relations. Seuls les concepts et relations portant `valide=true` sont parcourus. Le résultat contient les preuves, conditions, exceptions et statuts. Si aucune connexion n'existe, le système l'indique sans demander au LLM de compléter le chemin.

8.6 Recherche documentaire hybride

La recherche vectorielle calcule l'embedding de la question puis compare ce vecteur aux embeddings des chunks avec la similarité cosinus. En parallèle, l'index plein texte retrouve les expressions exactes. Les deux classements sont fusionnés par Reciprocal Rank Fusion :

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{60 + \mathrm{rang}_r(d)}.$$

Le score ne remplace pas les règles d'ancrage. Lorsqu'aucune classe n'est résolue, une correspondance lexicale minimale est exigée pour éviter qu'une simple proximité vectorielle ne ramène un passage hors sujet. Les chunks explicitement associés à une preuve métier sont placés en priorité et dédupliqués.

Le prototype de reranking de `ml/` calcule des caractéristiques telles que score sémantique, score lexical, score fusionné et recouvrement des mots. Il reste désactivé, car le faible jeu d'entraînement ne démontre pas un gain par rapport à la fusion déterministe.

8.7 Pipeline métier : de la fiche à Neo4j

8.7.1 Extraction contrôlée

`extraction_globale.py` découvre les fichiers `project_data/procedures/*.txt` et appelle `extraction_metier.py` document par document. Des checkpoints permettent de reprendre après interruption. Pour chaque morceau de texte, le programme sélectionne au maximum trente classes UML candidates. La sélection est calculée avant l'appel au LLM ; elle limite le contexte et interdit au modèle de proposer librement une classe inexistante.

Le prompt demande un JSON comportant des concepts, leurs alias, leur catégorie, leurs classes Arianes candidates et des triplets source–prédicat–cible. Chaque relation doit être accompagnée d'une citation exacte. La sortie reste une *proposition* : elle est écrite dans `project_data/metier/propositions/` et n'est pas importée automatiquement.

8.7.2 Validation V3 graduée

`validation_metier.py` applique cinq critères pondérés : preuve exacte (30 points), concepts identifiés (20), prédicat confirmé (20), direction confirmée (15) et condition

complète (15). Le score maximal est 100. Une erreur certaine, telle qu'une auto-relation, une extrémité absente ou une preuve invérifiable, conduit à `REJETE`. En l'absence de motif bloquant, un score d'au moins 85 avec passage disponible conduit à `VALIDE` ; les autres relations exploitables sont classées `PROBABLE`.

Point de vigilance. Dans ce rapport, `VALIDE` signifie confirmé par les contrôles documentaires. Il ne signifie pas certifié par un expert ferroviaire. Une relation `PROBABLE` est utilisable seulement avec une formulation prudente et ses conditions.

`recontrole_metier.py` relit les propositions à partir des passages sources et écrit une version V3 par document ainsi qu'une consolidation globale. `generer_pilotes.py` produit un pilote documentaire pour chaque fiche. `selection_prudente.py` réunit les relations `VALIDE` et `PROBABLE`, écarte les rejets et supprime les concepts non utilisés. Le fichier `pilote_global_documentaire.json` est l'entrée de l'import global.

8.7.3 Rôle des dossiers intermédiaires

— `propositions/` contient la sortie brute contrôlable du LLM ;

— `corrections_v2/` conserve l'ancien contrôle pour l'historique ;

— `corrections_v3/` contient le recontrôle gradué actuel ;

— `pilotes_documentaires/` contient une sélection exploitable par fiche ;

— `pilote_global_documentaire.json` consolide le corpus pour l'import ;

— `sauvegardes/` conserve l'état avant certaines fusions de doublons.

8.7.4 Import et traçabilité

`metier_builder.py` refuse les relations rejetées, fusionne les concepts par identifiant canonique et crée `RELATION_METIER`. Les propriétés comprennent le statut, le score, la condition, l'exception, les avertissements et les identifiants de chunks. Les ponts `REPRESENTE_PAR`, `DOCUMENTE_PAR` et `ETAYE_PAR` relient respectivement un concept à une classe UML, un document et ses preuves textuelles. La couche métier reste donc distincte du métamodèle tout en étant navigable dans la même base.

8.8 Construction de la réponse

`arianes/agent.py` ne décide pas quelles données lire : il reçoit le résultat du retriever. Pour un chemin, une première représentation déterministe est produite afin de figer les classes et relations. Pour une réponse générative, le contexte JSON contient uniquement les éléments retenus. Le prompt interdit l'ajout d'une classe, d'une relation ou d'une procédure absente et impose une formulation prudente pour `PROBABLE`.

L'utilisateur choisit explicitement Mistral, Gemini ou Groq dans l'interface. Il n'existe pas de basculement silencieux entre fournisseurs : une erreur 429 ou 413 est associée au fournisseur sélectionné. Ce choix facilite le diagnostic, car un résultat ne dépend pas d'un fallback invisible. Les embeddings documentaires restent actuellement calculés avec le service configuré lors de la construction et de la recherche.

8.9 API, authentification et mémoire

`api.py` initialise l'agent au démarrage, expose l'interface et stocke les comptes, sessions, conversations et messages dans SQLite. Les mots de passe sont dérivés avec PBKDF2-HMAC-SHA256, un sel et 600000 itérations. Le cookie de session est HttpOnly et SameSite strict. Chaque accès à une conversation vérifie son propriétaire.

La mémoire conversationnelle n'est pas stockée dans Neo4j. SQLite conserve l'historique par utilisateur ; le retriever ne réutilise le message précédent que pour les relances référentielles telles que « et ses attributs ? ». Une nouvelle question autonome ne doit pas être polluée par l'ancien sujet. Le graphe affiché dans l'interface provient du résultat de retrieval de la question et constitue un complément visuel à la réponse textuelle.

8.10 Reproductibilité et commandes

Listing 8.2 – Commandes principales depuis la racine

```
python -m arianes.pipelines.xmi_to_json
python -m arianes.graph.uml_builder
python -m arianes.pipelines.extraction_globale
python -m arianes.pipelines.recontrole_metier \
  --output-dir project_data/metier/corrections_v3
python -m arianes.pipelines.generer_pilotes
python -m arianes.pipelines.selection_prudente
python -m arianes.graph.metier_builder \
  project_data/metier/pilote_global_documentaire.json
python -m unittest discover -s tests -v
python -m uvicorn api:app --reload
```

La séparation des commandes permet de reconstruire seulement l'étape nécessaire et de contrôler chaque artefact avant l'import. Elle facilite également l'explication du projet : les scripts de préparation construisent une base traçable ; le retriever sélectionne ; le LLM formule ; l'API conserve la mémoire et applique les droits d'accès.

Chapitre 9

Validation et résultats

9.1 Stratégie de test

La validation distingue les fonctions unitaires, le retrieval, les règles de sécurité et l'intégration Neo4j. Les tests couvrent notamment :

- la résolution de classes et le choix du mode ;
- la vérification des citations ;
- les corrections directionnelles bornées ;
- le rejet des auto-relations ;
- la conservation de plusieurs preuves ;
- le refus d'importer une proposition non validée ;
- la récupération des chunks de preuve métier sans embedding ;
- l'isolation des conversations.

La dernière exécution effectuée pendant la rédaction a obtenu 73 tests réussis. Un avertissement de dépréciation Starlette reste à traiter lors d'une future mise à jour des dépendances.

9.2 Évaluation fonctionnelle du GraphRAG

Le module `arianes/evaluation.py` regroupe des scénarios couvrant les questions structurelles, documentaires, hybrides et hors domaine. L'évaluation du retrieval reste indépendante de la disponibilité ponctuelle du fournisseur de génération.

9.3 Pilote métier

Table 9.1 – État actuel de la couche métier dans Neo4j

Élément	Nombre
Concepts métier	255
Relations métier actives	347
Relations VALIDE	123
Relations PROBABLE	224
Relations REJETE importées	0

L'import repose sur des identifiants canoniques et sur `MERGE`. Les relations conservent leurs preuves et leurs statuts. Le retriever distingue ces relations métier des associations UML et peut calculer des chemins séparément dans les deux couches.

9.4 Discussion

Les résultats démontrent la faisabilité de la chaîne complète, depuis la source documentaire jusqu'à la réponse sourcée. Ils ne constituent pas une validation ferroviaire de l'ensemble des relations extraites. La précision d'une relation dépend de la qualité du passage, du schéma de prédicats, du prompt et de la relecture.

9.5 Limites

— la sélection initiale des classes candidates du pipeline métier est principalement lexicale ;

— les noms CamelCase et les synonymes peuvent réduire le rappel ;

— les relations `PROBABLE` exigent une revue experte ;

— le pilote global documentaire n'est pas une certification du corpus ;

— les réponses dépendent de la disponibilité du fournisseur choisi ;

— les métriques techniques ne remplacent pas une évaluation par des experts du domaine.

9.6 Prototype de reranking

Un prototype de reranking supervisé a été réalisé pour étudier une piste de machine learning. Avec seulement 40 exemples déséquilibrés, il a obtenu des résultats inférieurs à la

fusion hybride : nDCG@5 de 0,8725 contre 0,9369, et Recall@5 de 0,6367 contre 0,7033. Il reste donc désactivé. Cette expérience indique qu'un jeu de questions–chunks annoté par des experts est nécessaire avant toute activation.

Chapitre 10

Conclusion et perspectives

Le projet a transformé un squelette UML en un assistant GraphRAG complet. Le métamodèle Arianes est représenté dans Neo4j, les procédures sont découpées et indexées, puis un retriever contrôlé choisit les classes, relations et passages transmis au fournisseur sélectionné. L'application ajoute une mémoire conversationnelle et une séparation des données par utilisateur.

La couche métier constitue un second apport. Elle ne fusionne pas les concepts fonctionnels avec les classes UML, mais les relie en conservant leur provenance. Le pilote autour du poste d'aiguillage montre qu'une réponse peut distinguer une commande fonctionnelle d'une association du métamodèle et citer le passage qui la justifie.

La démarche de validation est volontairement progressive : les 29 fiches ont été extraites et le pilote documentaire global est actif, avec une séparation explicite entre relations confirmées documentairement et relations probables. Ce choix rend visible l'incertitude et prépare la validation experte.

Les perspectives prioritaires sont :

1. améliorer la sélection des classes candidates par CamelCase, synonymes et embeddings ;
2. créer une interface de revue des relations `PROBABLE` ;
3. faire annoter un jeu de questions et de preuves par des experts ;
4. faire certifier le graphe métier fiche par fiche par des experts ;
5. mesurer la fidélité des réponses aux citations ;
6. renforcer le déploiement : HTTPS, sessions, observabilité et base serveur ;
7. mesurer comparativement la qualité, le coût et la latence de Mistral, Gemini et Groq.

Résultat. L'assistant ne se limite plus à générer du texte : il identifie la classe, la relation et le passage qui justifient la réponse.

Annexe A

Fichiers, commandes et captures

A.1 Fichiers principaux

Table A.1 – Inventaire fonctionnel

Fichier	Fonction
`arianes/pipelines/xmi_to_json.py`	Conversion XMI vers JSON.
`arianes/graph/uml_builder.py`	Construction du graphe et indexation documentaire.
`arianes/retrieval.py`	Routage et retrieval UML, métier et documentaire.
`arianes/agent.py`	Génération contrôlée multi-fournisseur.
`api.py`	API, authentification, conversations et messages.
`index.html`	Interface du chatbot.
`arianes/pipelines/extraction_metier.py`	Extraction contrôlée d'une fiche.
`arianes/pipelines/extraction_globale.py`	Extraction des 29 fiches avec reprise.
`arianes/pipelines/validation_metier.py`	Contrôle gradué V3.
`arianes/pipelines/recontrole_metier.py`	Recontrôle hors ligne des checkpoints.
`arianes/pipelines/generer_pilotes.py`	Génération d'un pilote par fiche.
`arianes/pipelines/selection_prudente.py`	Sélection documentaire globale.
`arianes/graph/metier_builder.py`	Import protégé dans Neo4j.
`arianes/evaluation.py`	Scénarios d'évaluation fonctionnelle.

A.2 Commandes utiles

Listing A.1 – Tests et lancement de l'application

```
python -m unittest discover -s tests -v
python -m uvicorn api:app --host 127.0.0.1 --port 8000
```

Listing A.2 – Extraction et contrôle métier

```
python -m arianes.pipelines.extraction_globale
python -m arianes.pipelines.recontrole_metier --output-dir project_data/metier/
    corrections_v3
python -m arianes.pipelines.generer_pilotes
python -m arianes.pipelines.selection_prudente
python -m arianes.graph.metier_builder project_data/metier/
    pilote_global_documentaire.json
```

A.3 Requêtes Neo4j

Listing A.3 – Visualisation du pilote métier

```
MATCH (c:ConceptMetier)
OPTIONAL MATCH (c)-[r:RELATION_METIER]->(n:ConceptMetier)
RETURN c, r, n;
```

Listing A.4 – Visualisation de la classe Ligne

```
MATCH (c:Classe {nom: "Ligne"})
OPTIONAL MATCH (c)-[r]-(n)
RETURN c, r, n
LIMIT 40;
```

A.4 Captures à ajouter

Placer les images dans le dossier `figures/` avec les noms exacts indiqués dans `figures/README.md`. Le document compile sans elles et affiche des cadres réservés. Toute clé, tout mot de passe, cookie, jeton, identifiant personnel ou adresse interne doit être masqué avant insertion.

Bibliographie

[1] Alexis Chartrain. *Jumeau Numérique du système ferroviaire : proposition d'une approche de production par les ontologies et la modélisation de domaine. Intégration au Système d'Information de SNCF Réseau*. Thèse de doctorat, Université Clermont Auvergne, Clermont-Ferrand, 2026. Thèse préparée au LIMOS en collaboration avec SNCF Réseau dans le cadre d'une convention CIFRE.

[2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global : A graph rag approach to query-focused summarization. *arXiv preprint arXiv :2404.16130*, 2024.

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[4] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework : Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4) :333–389, 2009.